

Gather nodes are responsible for collecting back results from parallel execution nodes, assembling them together to produce the final result. The difference is that a **Gather Merge** node requires the parallel processes to provide it sorted output so that the assembling of the set of results is done following the ordering of the data.

A plain **Gather** node does not require the sorting of the batch results, so it simply assembles all the pieces together to provide the final result.

Parallel scans

All the main nodes that you can find in a sequential access method can be made parallel. Therefore, you can find a **Parallel Seq** scan, or index scans like **Parallel Index** and **Parallel Index-Only** scans and, of course, **Parallel Bitmap Heap** scans.

Parallel joins

When PostgreSQL decides to go for a parallel join method, it tries to keep the inner table accessed in a non-parallel way (assuming such a table is small enough) and performs parallel access to the outer table using one of the nodes presented in the last section.

However, in the case of **Hash Join**, the inner table is computed as a hash by every parallel process, which therefore requires every parallel process working on the outer table to compute the same results for the inner table. For that reason, there is also **Parallel Hash Join**, which allows a hash map of the inner table to be computed in parallel by every process working on the outer table.

Parallel aggregations

When the final result set is made by the aggregation of different parallel subqueries, there must be a parallel aggregation, which is the aggregation of every single parallel part.

This aggregation happens in different steps: first, there is a **Partial Aggregate** node, done by every parallel process that produces a partial result set. After that, a **Gather** node (or **Gather Merge**) collects all the partial results and passes the whole set to the **Finalize Aggregate** node, which sequentially assembles the final result.

When does the optimizer choose a parallel plan?

As already stated, PostgreSQL does not even consider a parallel plan as a choice if the expected size of the result set is small. In particular, if the table to seek data for has a dimension lower than the `min_parallel_table_scan_size` parameter (defaults to 8 MB), or the index to walk through is smaller than `min_parallel_index_scan_size` (defaults to 512 kB), PostgreSQL will not take into account a parallel plan at all.

You can force PostgreSQL to perform a parallel plan, even if the preceding values are not satisfied, with an extra configuration parameter – `debug_parallel_query` – which is set to `off` by default:

```
forumdb=> SHOW min_parallel_table_scan_size;
min_parallel_table_scan_size
-----
8MB
(1 row)

forumdb=> SHOW min_parallel_index_scan_size;
min_parallel_index_scan_size
-----
512kB
(1 row)

forumdb=> SHOW debug_parallel_query ;
debug_parallel_query
-----
off
(1 row)
```

In any case, when PostgreSQL considers the parallel plan to be an option, it does not default to using it: it rather evaluates carefully the costs of a sequential plan and the costs of the parallel plan to see whether it is still worth the extra setup effort.

There are, however, other restrictions to the application of a parallel plan: PostgreSQL must ensure not to spawn too many parallel processes, so if there are already too many parallel processes working on the system, the parallel execution will not be considered an option. Moreover, any statement that produces a data write – that is, anything different from a `SELECT` statement – will not be a valid candidate for a parallel plan, as well as any statement that can be suspended and resumed, such as the usage of a cursor.

Lastly, any query that involves the invocation of a function marked as `PARALLEL UNSAFE` will not produce a parallel plan candidate.

Utility nodes

Besides the already-introduced nodes that are used to access the data in a single table – or in multiple ones, in the case of joins – there are also some utility nodes that are used in a plan to achieve the final result.

When your statement involves an ordering of the result that is a clause such as `ORDER BY`, the planner inserts a **Sort** node. If the query has an output limitation, such as a `LIMIT` clause, a **Limit** node is inserted in the plan to reduce the final result set.

In that case, instead of a `UNION ALL` statement, the node used is an **Append** one (remember that `UNION ALL` allows duplicated tuples, while `UNION` does not).

If your statement involves the aggregation of different queries, like `UNION`, a **Distinct** node is inserted. The very same node has another feature: it can serve a `DISTINCT` tuple selection.

When a statement uses a `GROUP BY` clause, the planner inserts a **GroupAggregate** node responsible for the tuple squashing. Similarly, when the statement involves a window function (refer to *Chapter 7, Server-Side Programming*), the planner introduces a **WindowAgg** node for managing the tuple aggregation required by the window function.

In the case of a **Common Table Expression (CTE)**, the planner introduces a **CTEScan** node responsible for the join between the CTE subquery and the real table. If a join requires the materialization of a dataset – that is, if there is the need to simulate a table from a set of query results – the planner introduces a **Materialize** node.

Node costs

The PostgreSQL planner must evaluate the lowest cost execution plan, and in order to compute all possible alternatives, it computes the costs of all the evaluated access plans.

Every node is associated with a cost, which is the estimation of how expensive, in terms of computational resources, the execution of the node will be. Of course, every node has a variable cost that depends on the type and quantity of the input, as well as the node type.

PostgreSQL provides a list of costs, expressed in arbitrary units, for the main type of operations that a node can perform. Computing the cost of a node is, therefore, the computation of the cost of the single operations that the node performs multiplied by the number of times these operations are repeated, and this depends on the size of the data that the node has to evaluate.

The costs can be adjusted in the cluster configuration – that is, in the `postgresql.conf` main file or in the `pg_settings` catalog. In particular, it is possible to query the cluster about the main costs involved in a node execution:

```
forumbd=> SELECT name, setting
           FROM pg_settings
           WHERE name LIKE 'cpu%\_cost'
```

```
OR name LIKE '%page\_cost'
ORDER BY setting DESC;
```

name	setting
-----+-----	
random_page_cost	4
seq_page_cost	1
cpu_tuple_cost	0.01
cpu_index_tuple_cost	0.005
cpu_operator_cost	0.0025

The preceding are the default costs for a fresh installation of PostgreSQL, and you should not change any of the preceding values unless you are really sure about what you are doing. Remember that the costs are what make the planner choose between different plans, so setting the costs incorrectly will lead to the optimizer adopting the wrong execution plans.

Costs are expressed as “expenses,” but the values are not related to the time execution takes: the cost expresses the effort PostgreSQL has to expend to get the data; therefore, a higher cost must require a bigger effort.

As you can see from the preceding list of costs, the base for all the optimizer computations is the cost of a single data page accessed in sequential mode: this value is set to the unit of cost. CPU costs, which are costs related to the analysis of a tuple already in memory, are much smaller than a unit, while the access to the storage in a random way is much more expensive than sequential access.

Costs can change depending on the computation power of your system; in particular, having enterprise-level SSD storage disks can decrease your `random_page_cost` to 1.5, which is almost the same as a sequential page cost.

Changing the optimizer cost, although it is as simple as changing a few settings, requires a very deep knowledge of the PostgreSQL internals and of the underlying hardware, and is useful only in very specific cases; therefore, it is discouraged in pretty much all scenarios, and it is not within the scope of this book. Rather, you are going to understand how the planner estimates the costs of accessing the data.

Later in this chapter, you will see how the preceding costs are applied to compute the overall cost of a query plan.

In the following section, you will learn about indexes, the feature with which PostgreSQL can access your data more efficiently.

Indexes

An index is a data structure that allows faster access to the underlying table so that specific tuples can be found quickly. Here, “quickly” means faster than scanning the whole underlying table and analyzing every single tuple.

PostgreSQL supports different types of indexes, and not all types are optimal for every scenario and workload. In the following sections, you will discover the main types of indexes that PostgreSQL provides, but in any case, you can extend PostgreSQL with your own indexes or indexes provided by extensions.

An index in PostgreSQL can be built on a single column or multiple columns at once; PostgreSQL supports indexes with up to 32 columns.

An index can cover all the data in the underlying table, or can index specific values only – in that case, the index is known as “partial.” For example, you can decide to index only those values of certain columns that you are going to use the most.

An index can also be unique, meaning that it is used to ensure the uniqueness of the values it indexes, such as, for example, the primary keys of a table. Moreover, an index can be built on top of a user-defined function, which means the index is going to index the return values of those functions.



In order to be used in an index, a user-defined function must be declared as `IMMUTABLE`, which means its output must be the same for the very same input.

PostgreSQL is able to mix and match indexes together; therefore, multiple different indexes can be used to satisfy the query plan. Thanks to this important feature of PostgreSQL, you don’t have to define all the possible column permutation indexes, since PostgreSQL will try to mix unrelated indexes together.

In the following subsections, you will learn about all the available indexes types in a PostgreSQL 16 cluster, as well as how to create or drop an index.

Index types

The default index PostgreSQL uses is **Balanced Tree (B-Tree)**, a particular implementation of a tree that keeps its depth constant even with large increases in the size of the underlying table, therefore requiring the same effort to be traversed from the root to its leaves.

A B-Tree index can be used for most operators and column types, even string comparisons in LIKE-based queries, but it is effective only if the pattern starts with a fixed string. The B-Tree index also supports the UNIQUE condition and is therefore used to build the primary key indexes.

One drawback of the B-Tree index is that it copies the whole column's values into the tree structure; therefore, if you use a B-Tree to index large values (for example, long strings), the index will rapidly grow in size and space.

Another type of index that PostgreSQL provides is the **hash index**: this index is built on the result of a hash function for the value of the column(s). It is important to note that the hash index can be used only for equality operators, not for range nor disequality operators. In fact, being an index built on a hash function, the index cannot compare two hash values to understand their ordering; only the equality (which produces the very same hash value) can be evaluated.

Block Range Index (BRIN) is a particular type of index that is based on the range of values in data blocks on storage. The idea is that every block has a minimal and maximal value, and the index then stores a couple of values for every data block on the storage. When a particular value is searched from a query, the index knows in which data block the values can be found, but all the tuples in the block must be evaluated.

Therefore, this type of index is not as accurate as a B-Tree and is called *lossy* (to emphasize it is not exact; i.e., it can have losses), but it is much smaller in size with respect to all the other types of indexes since it only stores a couple of values for every data block.

GIN is a type of index that instead of pointing to a single tuple points to multiple values, and to some extent, to an array of values. Usually, this kind of index is used in full-text search scenarios, where you are indexing a written text where there are multiple duplicated keys (for example, the same word or term) that point to different places (for example, the same word in different phrases and lines).

Then comes **Generalized Index Search Tree (GIST)**, which is a platform on top of which new index types can be built. The idea is to provide a pluggable infrastructure where you can define operators and features that can index a data structure. An example is SP-GIST, a spatial index used in geographical applications.

Creating an index

Indexes can be created by means of the CREATE INDEX statement, which looks as follows:

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ] name ] ON [
ONLY ] table_name [ USING method ]
```

```
( { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [
ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ... ] )
[ INCLUDE ( column_name [, ...] ) ]
[ WITH ( storage_parameter = value [, ... ] ) ]
[ TABLESPACE tablespace_name ]
[ WHERE predicate ]
```

Indexes are identified by a mnemonic name, similar to the tables that they are related to. It is interesting to note that the index name is always unqualified, which means it does not include the schema where the index is going to live: an index is always found within the very same schema as the underlying table. However, it is possible to store an index in another tablespace than that of the underlying table, and this can be useful to store important indexes in faster storage. The statement supports the `IF NOT EXISTS` clause to abort the creation in a gentle way if an index with the same name already exists.

The `UNIQUE` clause specifies that the index is going to verify the uniqueness of its columns. The `WHERE` clause allows the creation of a partial index, which is an index that contains information only about those tuples that satisfy the `WHERE` condition(s).

The `INCLUDE` clause allows you to specify some extra columns of the underlying table that are going to be stored in the index, even if not indexed. The idea is that if the index is useful for an index-only scan, you can still get extra information without the trip to the underlying table. Of course, having a covering index (which is the name of an `INCLUDE` clause index) means that the index is going to grow in size and, at the same time, every tuple update could require extra index update effort.

The `USING` clause allows the specification of the type of index to be built, and if none is specified, the default B-Tree is used.

The `CONCURRENTLY` clause allows the creation of an index in a concurrent way: when an index is in its building phase, the underlying table is locked against changes so that the index can finish its job of indexing the tuple values. In a concurrent index creation, the table allows changes even during index creation, but once the index has been built, another pass on the underlying table is required to “adjust” what has changed in the meantime.

In order to make it more practical, let’s see how to build a simple index on the `posts` table. Let’s say we want to index the category a post belongs to:

```
forumdb=> CREATE INDEX idx_post_category
```

```
        ON posts( category );
CREATE INDEX
```

The preceding code will create an index named `idx_post_category` on the `posts` table, using the single-column `category` and the default index type (B-Tree).

The following does something similar, creating a multi-column index:

```
forumdb=> CREATE INDEX idx_author_created_on
           ON posts( author, created_on );
CREATE INDEX
```

It is important to note that, when creating multi-column indexes, you should always place the most selective columns first. PostgreSQL will consider a multi-column index from the first column onward, so if the first columns are the most selective, the index access method will be the cheapest. In the preceding example, assuming we want to search for a combination of authors and dates, we could expect many authors to publish on a specific day, so the date (the `created_on` column) is not going to be very selective, at least not as selective as the specific author; it is for that reason that we pushed the `created_on` column to the right in the column list.

If we would like to create a hash index, we could do something such as the following:

```
forumdb=> CREATE INDEX idx_post_created_on
           ON posts USING hash ( created_on );
CREATE INDEX
```

Of course, such an index will be useful only for equality comparison, so a query such as the following will never use the preceding index:

```
SELECT * FROM posts WHERE created_on < CURRENT_DATE;
```

But a query like the following could use the hash index:

```
SELECT * FROM posts WHERE created_on = CURRENT_DATE;
```

This is because we are asking for an equality comparison.

Inspecting indexes

Indexes are “attached” to their underlying tables, and so `psql` shows the defined indexes whenever you ask it to describe a table with the `\d` special command:

```
forumdb=> \d posts

Table "forum.posts"
```

```

      Column      |      Type      | Collation | Nullable |
Default
-----+-----+-----+-----+-----
pk                | integer        |           | not null |
generated always as identity
title            | text          |           |          |
content          | text          |           |          |
author           | integer       |           | not null |
category         | integer       |           | not null |
reply_to         | integer       |           |          |
created_on        | timestamp with time zone |           |          |
CURRENT_TIMESTAMP
last_edited_on   | timestamp with time zone |           |          |
CURRENT_TIMESTAMP
editable         | boolean       |           |          | true
likes            | integer       |           |          | 0
Indexes:
    "posts_pkey" PRIMARY KEY, btree (pk)
    "idx_author_created_on" btree (author, created_on)
    "idx_post_category" btree (category)
    "idx_post_created_on" hash (created_on)
...

```

As you can see from the preceding snippet of code, the command shows all the available indexes with their method (for example, btree) and a list of the columns the index is built on top of.

The `pg_index` special catalog contains information about the indexes and their main attributes, and so it can be queried to get the very information (and more) that is provided by `psql`. In particular, since an index is registered into `pg_class` with the special `relkind` value of `i`, we can join `pg_class` and `pg_index` to get detailed information in a statement, as follows:

```

forumdb=> SELECT relname, relpages, reltuples,
               i.indisunique, i.indisclustered, i.indisvalid,
               pg_catalog.pg_get_indexdef(i.indexrelid, 0, true)
FROM pg_class c JOIN pg_index i on c.oid = i.indrelid
WHERE c.relname = 'posts';

```

```

-[ RECORD 1 ]---+-----
relname      | posts
relpages     | 21
reltuples    | 1004
indisunique  | t
indisclustered | f
indisvalid   | t
pg_get_indexdef | CREATE UNIQUE INDEX posts_pkey ON posts USING btree (pk)
-[ RECORD 2 ]---+-----
relname      | posts
relpages     | 21
reltuples    | 1004
indisunique  | f
indisclustered | f
indisvalid   | t
pg_get_indexdef | CREATE INDEX idx_post_category ON posts USING btree
(category)
-[ RECORD 3 ]---+-----
relname      | posts
relpages     | 21
reltuples    | 1004
indisunique  | f
indisclustered | f
indisvalid   | t
pg_get_indexdef | CREATE INDEX idx_author_created_on ON posts USING btree
(author, created_on)
-[ RECORD 4 ]---+-----
relname      | posts
relpages     | 21
reltuples    | 1004
indisunique  | f
indisclustered | f

```